

Universidade de Brasília (UnB)
Departamento de Ciência da Computação

Disciplina: Sistemas Operacionais
Prof: Alba Melo
Período: 02/2026

Descrição do Trabalho Prático (02/2026)

Título: *Múltiplos Produtores / Múltiplos Consumidores*

1. Pré-requisitos

O aluno deverá conhecer a linguagem C e as chamadas de sistema Unix explicadas no curso.

2. Descricão

O presente trabalho consiste da implementação de um sistema com múltiplos produtores e múltiplos consumidores em (a) memória compartilhada e (b) troca de mensagens:

```
> mpmc p paradigma &
```

onde *p* é o número de produtores e *paradigma* é memória compartilhada (*paradigma=1)* ou troca de mensagem (*paradigma=2)*. Caso o paradigma seja 1, os mecanismos IPC *shm* e *sem* devem ser usados. O tamanho do segmento de memória criado deve ser 2048 bytes. Caso o paradigma seja 2, o mecanismo IPC *msg* deve ser usado. Adicionalmente, os sinais Unix podem ser usados também. A estrutura de dados da mensagen deve conter: pid do processo que a enviou, número sequencial da mensagem para o processo emissor, tempo do envio.

O processo *mpmc* fica esperando comandos no shell com ">>"

Para o lançamento do mpmc pode ser usado "&" ou não.

Os produtores enviam dados com o comando para o mpmc:

```
>> produz p n_msgs ack &
```

onde *p* é o produtor e *n_msgs* é o número de mensagens (de 1 a 1024). Caso *p* seja especificado como zero, as *n_msgs* serão enviadas no padrão striped (1, 2, …, p, 1, 2, …, p). O parâmetro *ack* determina se o produtor deve esperar a confirmação de que todas as mensagens foram consumidas (1 – sim, 2 – não).

O processo *produz* é criado e ele produz as mensagens, colocando-as uma a uma na fila ou na memória compartilhada. O processo *produz* opera de maneira concorrente com o processo *mpmc*, que volta a aceitar comandos com ">>"

Os consumidores recebem dados com o comando:

```
>> consome p n_msgs ack &
```

onde *p* é o produtor do qual o dado deve ser consumido, *n_msgs* é o número de mensagens a serem consumidas (1 a 1024). Note que cada consumidor pode consumir mais de uma mensagem no mesmo comando. No caso de haver menos mensagens do que *n_msgs*, o consumidor fica bloqueado se *ack=1* ou um erro é retornado, se *ack=2*, informando o número de mensagens que foram consumidas.

O processo *consome* é criado e ele consome as mensagens (uma a uma) da fila ou da memória compartilhada. O processo *consome* opera de maneira concorrente com o processo *mpmc*, que volta a aceitar comandos com ">>"

O sistema é terminado com o comando:

```
>> shutdown
```

onde os processos produtores e consumidores, bem como o processo *mpmc* são terminados, o tempo de execução de cada processo é impresso e os mecanismos de comunicação são deletados. O número total de mensagens consumidas em cada estrutura *p* e o tempo de operação do *mpmc* são impressos.

3. Exemplos

3.1 Quatro Produtores em memória compartilhada

```
> mpmc 4 1 &
>> produz 3 2 1 &
>> consome 2 2 1 &
>> produz 2 5 1 &
>> shutdown
```

São criados 4 produtores, com as estruturas de dados em memória compartilhada, ou seja, 4 memórias compartilhadas de tamanho 2048 bytes. O produtor 3 produz duas mensagens na memória compartilhada 2 e fica bloqueado esperando a confirmação de que elas foram consumidas. O consumidor pede para consumir 2 mensagens da memória compartilhada 2 e fica bloqueado enquanto as duas mensagens não tiverem saído do buffer. O produtor 2 produz 5 mensagens na memória compartilhada 2 e é bloqueado. Quando duas mensagens forem produzidas, o consumidor consome as mensagens. A execução é finalizada com o produtor bloqueado. O número de mensagens consumidas e o tempo de operação são impressos.

3.2 Dois Produtores em troca de mensagem

```
> mpmc 2 2 &
>> produz 3 2 1 &
>>>> retorna erro – fila 3 nao existe
>> consome 2 2 1 &
>> consome 2 2 1 &
>> produz 2 5 2 &
>> shutdown
```

São criados 2 produtores, com as estruturas de dados em fila de mensagem. O produtor produz duas mensagens na fila de mensagens 3, mas não existem 3 filas de mensagens, então um erro é retornado. O primeiro consumidor pede para consumir 2 mensagens da fila de mensagens 2 e fica bloqueado enquanto as duas mensagens não tiverem saído do buffer. O segundo consumidor pede para consumir 2 mensagens da fila de mensagens 2 e fica bloqueado enquanto as duas mensagens não tiverem saído do buffer. O produtor 2 produz 5 mensagens na fila de mensagens 2 e não fica bloqueado. A partir daí, duas mensagens são consumidas pelo primeiro consumidor, que termina a execução, e duas mensagens são consumidas pelo segundo consumidor, que termina a execução. A execução é finalizada. O número de mensagens consumidas e o tempo de operação são impressos.

3.2 Cinco Produtores em troca de mensagem

```
> mpmc 5 2 &
>> produz 0 15 2 &
>> consome 2 3 1 &
>> consome 1 2 1 &
>> produz 3 2 1 &
>> shutdown
```

São criadas 5 filas de mensagem. São produzidas 15 mensagens, colocadas nas 5 filas no padrão striped (3 em cada fila). O primeiro consumidor pede para consumir 3 mensagens da fila 2 e fica bloqueado enquanto as duas mensagens não tiverem saído do buffer. O segundo consumidor pede para consumir 2 mensagens da fila 1 e fica bloqueado enquanto as duas mensagens não tiverem saído do buffer. O produtor produz 2 mensagens na fila 3 e fica bloqueado. A execução é finalizada com o produtor bloqueado. O número de mensagens consumidas é impresso.

BOM TRABALHO !!!!!